# CIS 371 Web Application Programming

## Cloud DataBase

### Firebase Cloud Firestore

GRAND VALLEY STATE UNIVERSITY


Lecturer: Dr. Haoyu Li



Based on the original version by Professor Hans Dulimarta and Professor Yong Zhuang.

# Exercise from the previous class

## Practice: Responsive Product Card Dashboard

Requirements

- Create v-cols for all products defined in the script.
- Each v-col should contain a v-card showing:
  - name, price, and a Buy button
- Use Vuetify's grid system to make the layout responsive:
  - 4 columns per row on large screens
  - 2 columns per row on medium screens
  - 1 column per row on small screens

Answer

# Practice

- Create v-cols for all products defined in the script.
- Each v-col should contain a v-card showing:
  - name, price, and a Buy button
- Use Vuetify's grid system to make the layout responsive:
  - 4 columns per row on large screens
  - 2 columns per row on medium screens
  - 1 column per row on small screens

```
<v-row>
<v-col
v-for="(product, index) in products"
:key="index"
cols="12"
sm="12"
md="6"
lg="3"
>
<v-card class="pa-3" outlined>
<v-card-title class="text-h6">{{ product.name }}</v-card-title>
<v-card-subtitle>${{ product.price.toFixed(2) }}</v-card-subtitle>
<v-card-actions>
<v-btn color="primary">Buy</v-btn>
</v-card-actions>
</v-card>
</v-col>
</v-row>
```

# Why Cloud Data Stores?

- Highly scalable
- Usually built using No SQL technology
- Accessible to both web and mobile clients

# No SQL = No DB Schema

# SQL vs. noSQL

- Relational model
- Schema: relationship between tables and fields
- Popular examples
    - Oracle
    - DB2
    - MySQL
    - PostGreSQL

- Non-relational
- Schemaless Datastore
- Cloud Computing and Cloud Storage
- Rapid Development
- Popular examples
    - MongoDB
    - CouchDB
    - BigTable
    - Firebase Realtime DB
    - Firebase Cloud Firestore

# Schema or Schemaless?

| First | Last | G# | Major |
|---|---|---|---|
| Alice | Smith | 12345678 | Statistics |
| Brad | Jordan | 23456789 | History |

*Must redefine the SCHEMA to add a new column.*

| First | Last | G# | Major | Color | SocMedia | ? |
|---|---|---|---|---|---|---|
| Alice | Smith | 12345678 | Statistics | Green | | |
| Brad | Jordan | 23456789 | History | | IG, FB | |
| Gary | deGroot | 72551834 | Biology | | TW | |
| Ann | Hunt | 78921631 | Physics | Blue | | |
| Fay | Ross | 72631235 | English | | LinkedIn | |

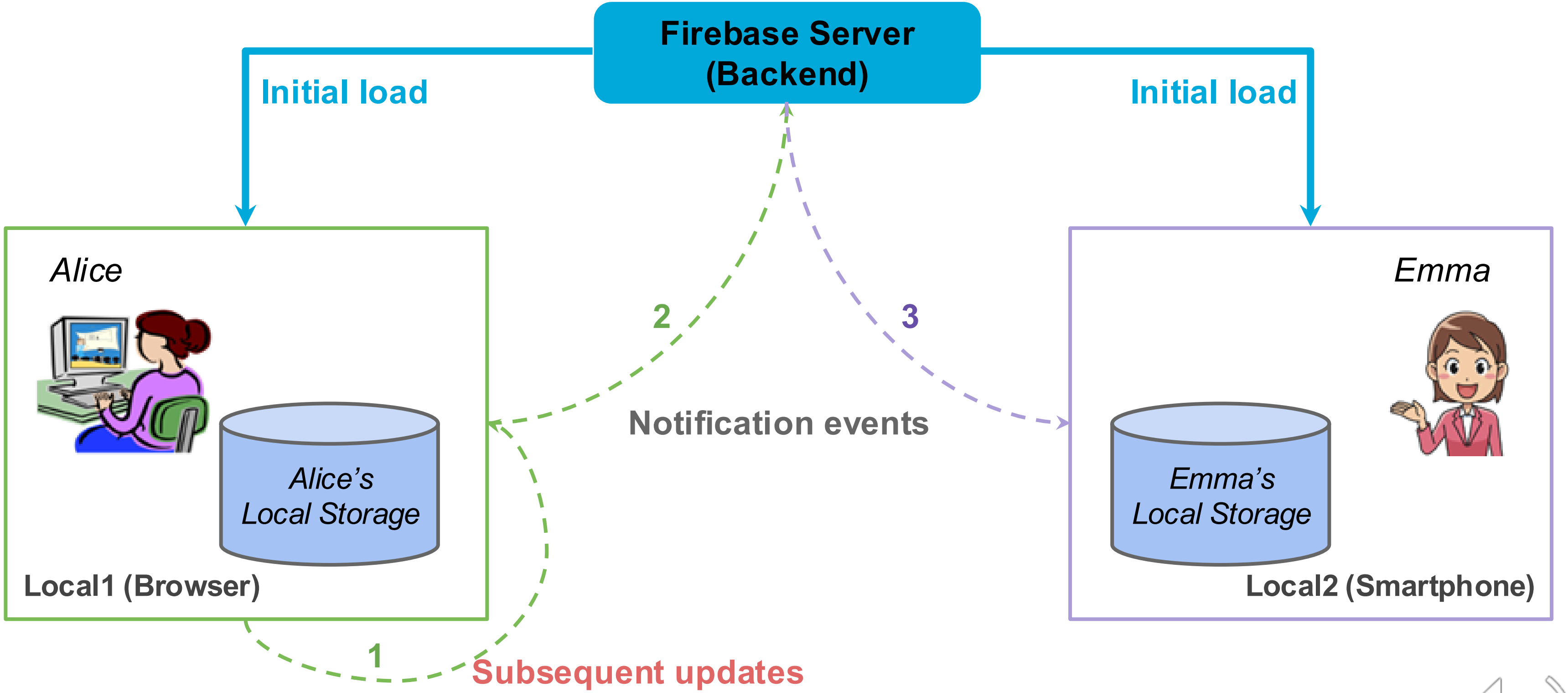
Grand Valley State University

# Firebase: A collection of many products

- Authentication
- Cloud Firestore
- Cloud Storage
- Cloud Functions
- …

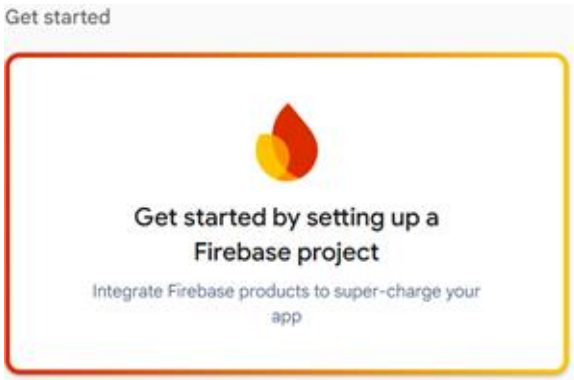
Grand Valley State University

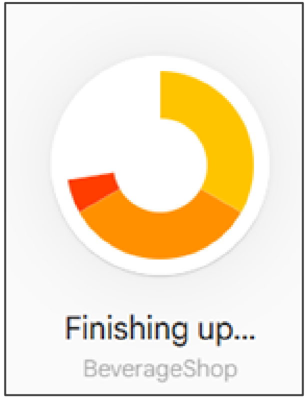
# Local Storage, Local Events, & Global Events



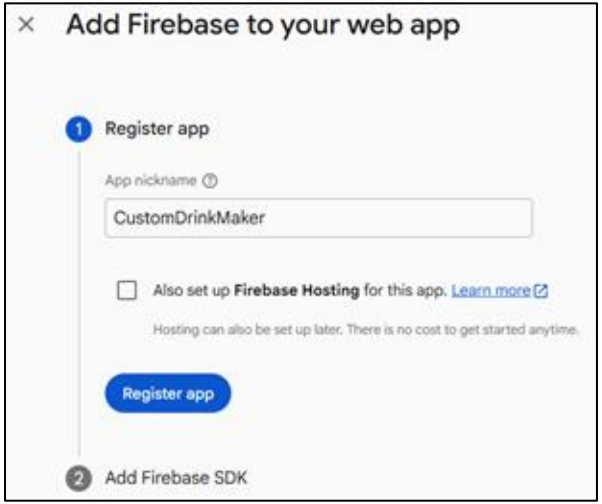
# Creating a new WebApp

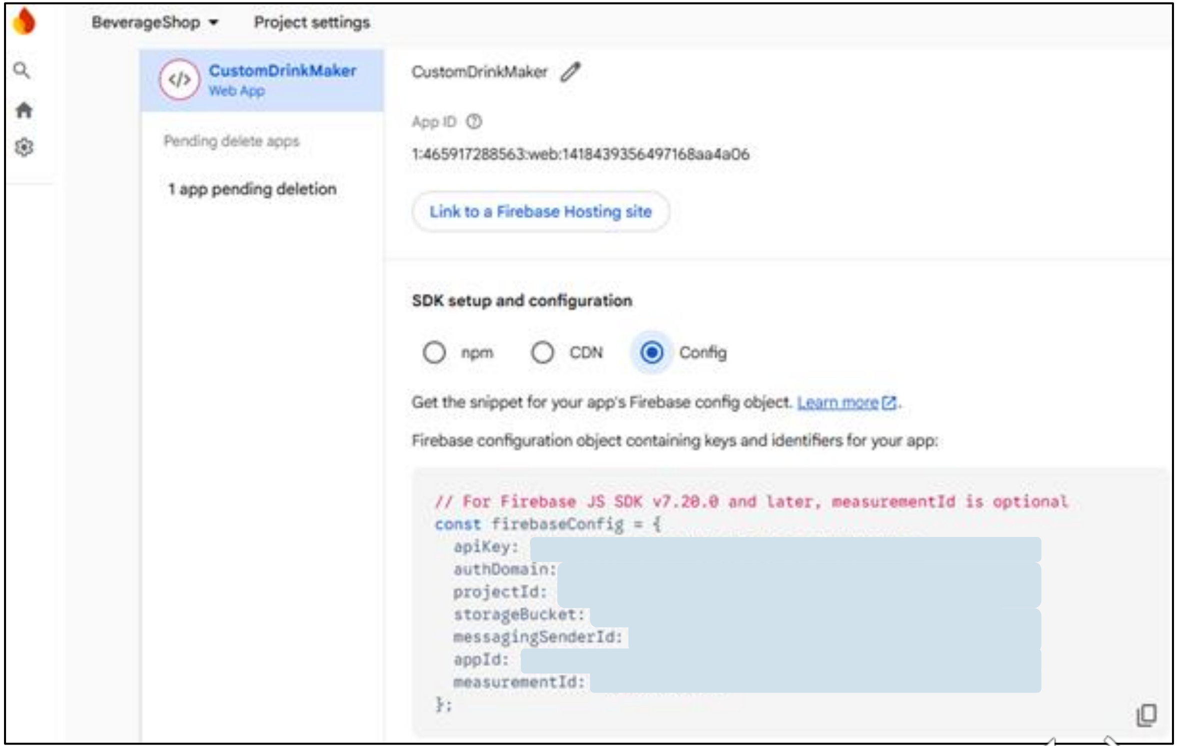
1. Use a Google account to login to [Firebase Console]
2. Create a new project
3. Add a webapp to the project



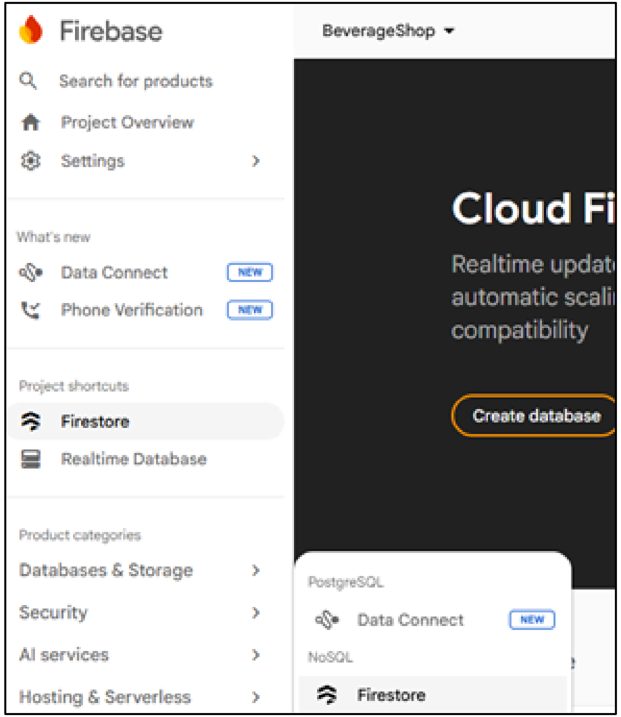



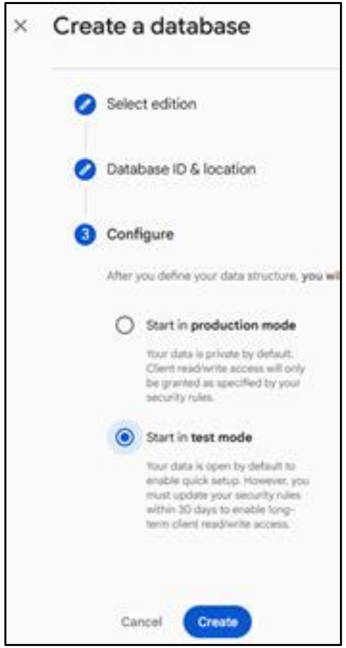

# Creating a new WebApp



# Initialize Firestore



# Local Project Setup
## (On Your Computer)

# Project Setup & Initialization

```
yarn init -y
yarn add firebase
```

**OR**

```
npm init -y
npm install firebase
```

```
import { initializeApp, FirebaseApp } from "firebase/app";
import { getFirestore, Firestore } from "firebase/firestore";

const firebaseConfig = {
  apiKey: "your-api-key-goes-here",
  authDomain: "your-project-name-here.firebaseapp.com",
  databaseURL: "https://your-project-name-here.firebaseio.com",
  projectId: "your-project-name-here",
  storageBucket: "your-project-name.appspot.com",
  messagingSenderId: "xxxxxxxx"
};

// Initialize Firebase
const myapp: FirebaseApp = initializeApp(firebaseConfig);
const db: Firestore = getFirestore(myapp);
```

*// COPY this from your Firebase Console*

**Initialize Cloud Firestore**

# Database Dashboard

- Browse and Modify Data
- Security Rules (default settings: user authentication required)

```
// Allow read/write access on all documents to any user signed in to the application
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} {
      allow read, write: if request.auth != null;
    }
  }
}
```

Cloud Firestore Security Rules

# Data Model: Hierarchy of Collections-Documents

- Hierarchical structure
  - The “root” holds one or more collections
  - A collection consists of one or more documents
  - A document is one or more key-value pairs
  - A value in a document may refer to a subcollection (1-to-many relationships)
- Data Types in a document
  - string, number, boolean, array, timestamp, map (kv-pairs), geolocation
  - Reference to a subcollection

| SQL | Cloud Firestore |
|---|---|
| Tables | Collections |
| Rows | Documents |
| Primary Key | Document ID |
| Fields | key-value pairs |

# Data Model: Hierarchy of Collections-Documents

**State (SQL table)**

| Abbrev (PK) | Name | Capital |
|---|---|---|
| AK | Alaska | Juneau |
| AL | Alabama | Montgomery |
| FL | Florida | Tallahassee |

**States (Collection of 3 Documents)**

**AK**

*Name: Alaska*
*Capital: Juneau*

**AL**

*Name: Alabama*
*Capital: Montgomery*

**FL**

*Name: Florida*
*Capital: Tallahassee*

**NatlPark (SQL table)**

| Code (PK) | Name | Location |
|---|---|---|
| U6123 | Arches | Utah |
| C1632 | Black Canyon | Colorado |

**Parks (Collection of 2 Documents)**

**U6123**

*Name: Arches*
*Location: Utah*

**C1632**

*Name: Black Canyon*
*Location: Colorado*

*SQL Table ⇒ Firestore Collection*
*SQL Primary Key ⇒ Firestore Document ID/"name"*
*SQL Table Row ⇒ Firestore Document*

# All Firestore Collection/Doc Manipulation Functions return a Promise

# Firestore Functions

- Functions for creating references
  - collection(refToFirestore, “path/to/collection”)
  - doc(refToFirestoreOrCollection, “path/to/your/document”)
  - query(refToCollection, ____)
- **Retrieval functions**
  - getDoc(refToDoc)
  - getDocs(refToCollection)
- **Manipulation functions**
  - addDoc(refToColl, { new_content_object })
  - setDoc(refToDoc, { new_content_object })
  - updateDoc(refToDoc, { new_content_object })
  - deleteDoc(refToDoc)
- Update listener `onSnapShot()` **(specific to Firebase)**

**C**
**R**
**U**
**D**

GRAND VALLEY STATE UNIVERSITY

# CRUD Operations (Summary)

| | Collection | Document |
|---|---|---|
| Create | Implied when a doc is created | `// Option #1`<br>`const collPar = collection(db, "cName");`<br>`addDoc(collPar, { /* new content here */ });`<br>`// Option #2`<br>`const myDoc = doc(db, "cName", "docName");`<br>`setDoc(myDoc, { /* new content here */ })` |
| Read | `const myC =`<br>`collection(db,"cName");`<br>`getDocs(myC).then(___);` | `const myDoc = doc(____, ____, ___);`<br>`getDoc(myDoc).then(___);` |
| Update | N/A | `const myDoc = doc(____, ____, ___);`<br>`updateDoc(myDoc, {/* content */}).then(___);` |
| Delete | N/A | `const myDoc = doc(____, ____, ___);`<br>`deleteDoc(myDoc).then(___);` |

GRAND VALLEY STATE UNIVERSITY

# CRUD Operations: Create Doc (own Doc ID)

SQL

```
// Use "AK" as the primary key for the tuple
INSERT INTO states (abbrev, name, capital) VALUES("AK", "Alaska", "Juneau")
```

**states (SQL table)**

| Abbrev (PK) | Name | Capital |
|---|---|---|
| AK | Alaska | Juneau |
| AL | Alabama | Montgomery |
| FL | Florida | Tallahassee |

```ts
import { DocumentReference, setDoc, doc } from "firebase/firestore";
// Option #1: Use file name syntax for doc path
// Primary key "AK" becomes doc id
const doc1: DocumentReference = doc(db, "states", "AK");
setDoc(doc1, { name: "Alaska", capital: "Juneau" })
  .then(() => {
    console.log("New doc added");
  })
  .catch((err: any) => {
    /* your code here */
  });
```

*Firestore in TS*

# CRUD Operations: Create Doc (automatic Doc ID)

INSERT INTO states (name, capital) VALUES(“Alaska”, “Juneau”)

SQL

**states (SQL table)**

| Name | Capital |
|---|---|
| Alaska | Juneau |
| Alabama | Montgomery |
| Florida | Tallahassee |

```
import { CollectionReference, addDoc, doc } from "firebase/firestore";

const myColl: CollectionReference = collection(db, "states");
addDoc(myColl, { name: "Alaska", capital: "Juneau" })
  .then(() => {
    console.log("New doc added");
  })
  .catch((err: any) => {
    /* your code here */
  });
```

*Firestore in TS*

# CRUD Operations: Create Docs from Array

*Firestore in TS*

```
import {
  DocumentReference,
  setDoc,
  doc,
  collection,
  addDoc,
} from "firebase/firestore";
const stateArr = [
  { abbrev: "CA", name: "California", capital: "Sacramento" },
  { abbrev: "CO", name: "Colorado", capital: "Denver" },
  // more data here
];
// Option 1: Use state abbreviation as document ID
stateArr.forEach(async (st: any) => {
  const stateDoc = doc(db, "states", st.abbrev); // Us Abbreviation as document ID
  await setDoc(stateDoc, { name: st.name, capital: st.capital });
});
// Option 2: Let Firestore generates automatic
const myStateColl = collection(db, "states"); // Do this outside .forEach
stateArr.forEach(async (st: any) => {
  await addDoc(myStateColl, { name: st.name, capital: st.capital });
});
```

await vs. .then()

# CRUD Operations: Read All Documents

SELECT * FROM states | SQL

states (SQL table)

| Abbrev (PK) | Name | Capital |
|---|---|---|
| AK | Alaska | Juneau |
| AL | Alabama | Montgomery |
| FL | Florida | Tallahassee |

```
// Assume saved data has the
// following structure
type StateType = {
  abbrev: string;
  name: string;
  capital: string;
};
```

```
import {
  CollectionReference,
  collection,
  QuerySnapshot,
  QueryDocumentSnapshot,
  getDocs,
} from "firebase/firestore";

const myStateColl: CollectionReference = collection(db, "states");

getDocs(myStateColl).then((qs: QuerySnapshot) => {
  qs.forEach((qd: QueryDocumentSnapshot) => {
    const stateData = qd.data() as StateType;
    const docId = qd.id; // Fixed 'cost' to 'const'
    // More code here to manipulate stateData
  });
});
```

*Firestore in TS*

# CRUD Operations: Read A Specific Document

SQL

```
// Select a tuple with a known primary key
SELECT * FROM states WHERE abbrev = "FL"
```

**states (SQL table)**

| Abbrev (PK) | Name | Capital |
|---|---|---|
| AK | Alaska | Juneau |
| AL | Alabama | Montgomery |
| FL | Florida | Tallahassee |

```
// Assume saved data has the
// following structure
type StateType = {
  abbrev: string;
  name: string;
  capital: string;
};
```

```
import {
  DocumentReference,
  doc,
  DocumentSnapshot,
  getDoc,
} from "firebase/firestore";
// FL is a document ID
const myDoc: DocumentReference = doc(db, "states/FL");
getDoc(myDoc).then((qd: DocumentSnapshot) => {
  if (qd.exists()) {
    const stateData = qd.data() as StateType;
    // More code here to manipulate stateData
  }
});
```

*Firestore in TS*

# CRUD Operations: Fetch Document(s) Where...

SQL

```
// Select tuples satisfying some conditions
SELECT * FROM states WHERE name = "Florida"
```

**states (SQL table)**

| Abbrev (PK) | Name | Capital |
|---|---|---|
| AK | Alaska | Juneau |
| AL | Alabama | Montgomery |
| FL | Florida | Tallahassee |

```
// Assume saved data has the
// following structure
type StateType = {
  abbrev: string;
  name: string;
  capital: string;
};
```

```
import {
  Query,
  getDocs,
  collection,
  where,
  query,
  QuerySnapshot,
  QueryDocumentSnapshot,
} from "firebase/firestore";
const getFL: Query = query(
  collection(db, "states"),
  where("name", "==", "Florida")
);
getDocs(getFL).then((qs: QuerySnapshot) => {
  qs.forEach((qd: QueryDocumentSnapshot) => {
    const stateData = qd.data() as StateType;
    // More code here to manipulate stateData
  });
});
```

*Firestore in TS*

# CRUD Operations: Fetch Document(s) Where...

SQL

```
// Select tuples satisfying some conditions
SELECT * FROM states WHERE population > 10_000_000
```

**states (SQL table)**

| Name | Capital | Population |
|---|---|---|
| California | Sacramento | 39_123_612 |
| Michigan | Lansing | 8_432_911 |
| Florida | Tallahassee | 26_222_943 |

```
// Assume saved data has the
// following structure
type StateType = {
  name: string;
  capital: string;
  population: number;
};
```

```
import {
  Query,
  getDocs,
  collection,
  query,
  where,
  QuerySnapshot,
  QueryDocumentSnapshot,
} from "firebase/firestore";
const aboveTenMil: Query = query(
  collection(db, "states"),
  where("population", ">", 10_000_000)
);
getDocs(aboveTenMil).then((qs: QuerySnapshot) => {
  qs.forEach((qd: QueryDocumentSnapshot) => {
    const stateData = qd.data() as StateType;
    // More code here to manipulate stateData
  });
});
```

Firestore in TS

# CRUD Operations: Fetch Document(s) Where...

SQL

// Select tuples satisfying some conditions
SELECT * FROM states WHERE population > 10_000_000
AND population < 15_000_000

**states (SQL table)**

| Name | Capital | Population |
| --- | --- | --- |
| California | Sacramento | 39_123_612 |
| Michigan | Lansing | 8_432_911 |
| Florida | Tallahassee | 26_222_943 |

```
// Assume saved data has the
// following structure
type StateType = {
  name: string;
  capital: string;
  population: number;
};
```

```
import {
  Query,
  getDocs,
  collection,
  query,
  where,
  QuerySnapshot,
  QueryDocumentSnapshot,
} from "firebase/firestore";
const aboveTenMil: Query = query(
  collection(db, "states"),
  where("population", ">", 10_000_000),
  where("population", "<", 15_000_000)
);
getDocs(aboveTenMil).then((qs: QuerySnapshot) => {
  qs.forEach((qd: QueryDocumentSnapshot) => {
    const stateData = qd.data() as StateType;
    // More code here to manipulate stateData
  });
});
```

*Firestore in TS*

# Available Query Where Operators

| Operator | Example | SQL Equivalent |
|---|---|---|
| <, <=, ==, >=, > | where(“population”, “>”, 20_000_000) | WHERE population > 20000000 |
| != | where(“name”, “!=”, “Andy”) | WHERE name != “Andy” |
| in | where(“city”, “in”, [“Ada”, “Flint”]) | WHERE city == “Ada” OR city == “Flint” |
| not-in | where(“city”, “not-in”, [“Ada”, “Flint”]) | WHERE city != “Ada” AND city != “Flint” |

| Operator | Example (courses must be an ARRAY) |
|---|---|
| array-contains | // Has this student taken MTH200? where(“courses”, “array-contains”, “MTH200”) |
| array-contains-any | // Has this student taken either MTH200 or STA215? where(“courses”, “array-contains-any”, [“MTH200”, “STA215”]) |

# Query Limitations (only for older version)

```
// Multiple .where() on the same field
const q = query(
  collection(__, "states"),
  where("population", ">=", 5_000_000),
  where("population", "<=", 10_000_000)
);
getDocs(q).then(() => {
  /* code */
});
```

***OK***

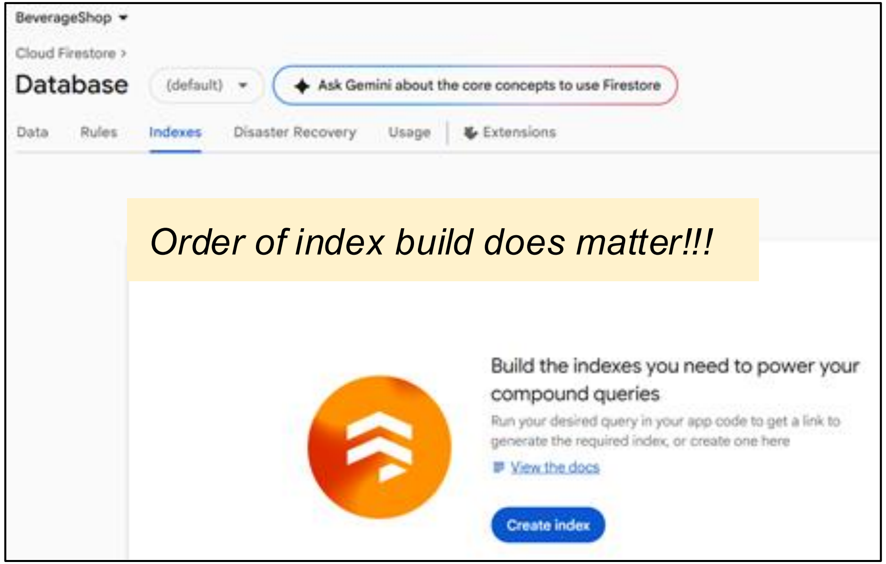
```
// Can"t use inequalities on two different fields
query(
  collection(__, "states"),
  where("population", ">=", 5_000_000),
  where("area", "<=", 200_000)
);
```

***Not OK***

```
// Multiple .where on different fields
// require a composite index on both fields
// At most one inequality comparison!!
const q = query(
  collection(__, "students"),
  where("major", "==", "MATH"),
  where("gpa", ">=", 3.0)
);
getDocs(q).then(/* more code */);
```

***OK***

# Building Composite Index




*Order of index build does matter!!!*

```
// Multiple .where on different fields
// require a composite index on both fields
// At most one inequality comparison!!
const q = query(
  collection(__, "students"),
  where("major", "==", "MATH"),
  where("gpa", ">=", 3.0)
);
getDocs(q).then(/* more code */);
```

# Exercises: Try Your First App with Firestore

**1: Create a Firebase Project**

- Go to Firebase Console, click **"Create a Firebase project"**
- Name your project **BeverageShop,** disable Google Analytics (optional), then click **Continue**

**2. Register Your App**

- In the **BeverageShop** project dashboard, click **“</>”** to add a **Web App**
- Name your app **BrewInTheCloud**
- Click **Register App**
- Follow the instructions to **add Firebase SDK** to your web app (copy the config snippet)

**3: Set Up Firestore**

- In the left sidebar, go to **"Build"** → **Firestore Database**
- Click **Create Database**
- Choose your Firestore location (United States)
- Start in **Test Mode** (for development)

**Ready to Code!**

- *Initialize Firebase in your vue3 app using the provided config*
- *Test your connection and start reading/writing data from Firest*